

# fedctl

**A tool for shared realistic edge-FL experiments in Flower**

Samuel Jie

Department of Computer Science and Technology  
University of Cambridge  
`fedctl.cl.cam.ac.uk`

Flower Monthly | 6 May 2026



# Federated learning runs on real systems

---

## What FL is trying to handle

- Data often cannot move because of privacy, governance, bandwidth, or ownership.
- The client is a real device, not just a statistical partition.
- Availability, CPU speed, memory pressure, and network path become part of training.

## Same algorithm

different devices, different timing

## Same dataset split

different update arrival order

## Same final accuracy

different wall-clock cost

# Heterogeneity changes the training loop

---

## Compute heterogeneity

- Devices have different CPU and memory budgets.
- Smaller devices may need smaller local models.
- The bottleneck shows up as training time and deployed-model quality.

## Network and availability

- Updates arrive at different times.
- Slow clients can contribute less often or with more staleness.
- The bottleneck shows up as time-to-target and client influence.

**fedctl makes these deployment conditions explicit instead of accidental.**

# Simulation hides deployment effects

---

## Convenient simulation

- Clients run as local processes.
- Timing is either homogeneous or injected as a model.
- Failures and logs disappear with the process.

## Real edge-style runs

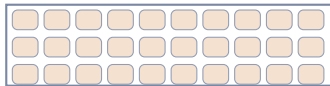
- Clients run on different hardware.
- Slow devices, bandwidth limits, and queueing are observable.
- Every run leaves inspectable evidence.

**The question becomes: how do we let multiple researchers use this kind of testbed without turning every run into cluster ops?**

# A shared Raspberry Pi testbed is feasible

---

Current pool: 50 Raspberry Pi workers



30 x rpi4



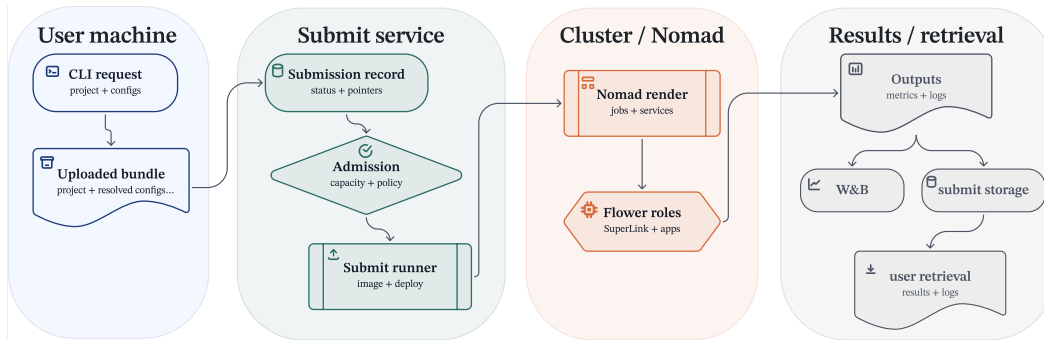
20 x rpi5

Researchers can request the full pool or a typed slice;  
device type is a deployment variable.

## What is hard

- Queueing shared access.
- Pinning logical clients to hardware classes.
- Repeating runs with the same conditions.
- Finding logs and artifacts after something fails.

# Submitting Flower projects to the cluster



## Submit

package a Flower project

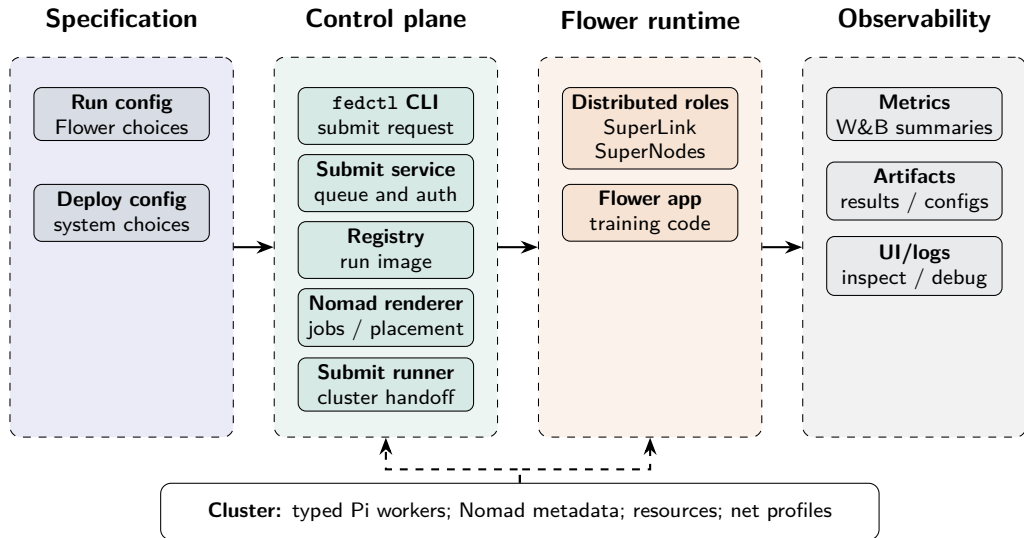
## Deploy

render Nomad jobs and placement

## Inspect

logs, jobs, metrics, artifacts

# High-level execution stack



# Two configs keep the experiment and the system separate

---

## Run config

### Flower-facing choices

- task, model, data partitioner
- method and hyperparameters
- client budget, seed, evaluation

## Deploy config

### fedctl-facing choices

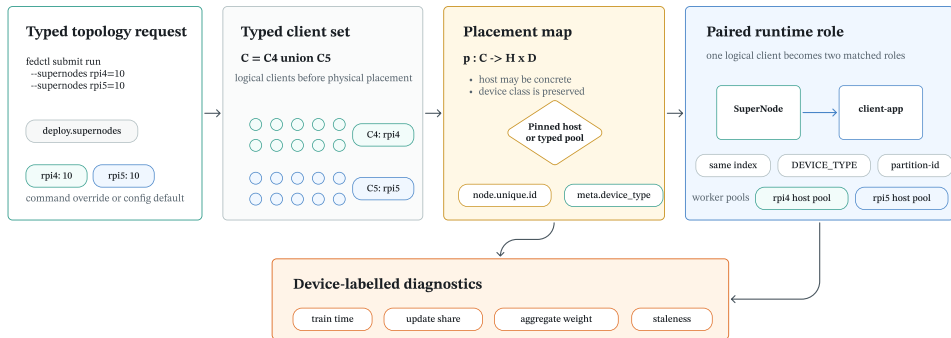
- rpi4=30, rpi5=20
- resource and placement policy
- network profile and registry

## Why this matters

The same learning experiment can run on a different topology, and the same topology can run different learning methods.



# Placement and network conditions are explicit



## Placement Network profiles

logical clients are mapped onto rpi4/rpi5 workers and recorded in metrics.  
delay, jitter, loss, and bandwidth caps can be assigned per logical client.

# What fedctl makes possible

---

## Use real devices

- Start from a normal Flower project.
- Run on Raspberry Pi workers instead of local processes.
- Keep the command-line workflow familiar.

## Share the cluster

- Queue submissions across users.
- Request typed slices from a 30 RPi4 + 20 RPi5 pool.
- Avoid hand-written scheduler jobs.

## Change conditions

- Fix placement and resources.
- Assign network profiles.
- Preserve logs, jobs, configs, and artifacts.

**The useful part is not owning a cluster; it is making it usable as a repeatable Flower workflow.**

# Demo

```
fedctl submit run ...
```

```
fedctl.cl.cam.ac.uk
```